

ImageAudit

Project Proposal

Version 1.0.0

Local-first dataset quality and diagnostics
for computer-vision datasets.

Project	ImageAudit
Version	1.0.0 (first stable public release)
Domain	Computer-vision dataset quality tooling
Architecture	Modular audit engine + FastAPI + Next.js UI
Backend	Python 3.11+, FastAPI, core audit library
Frontend	Next.js 14 (static export for packaging)
Deployment	Source install; Windows x64 standalone EXE
License	MIT
Repository	https://github.com/AmirSam-Eftekhari/ImageAudit

Project at a Glance

Technical proposal describing problem context, architecture, methodology, security model, validation evidence, limitations, and future directions.

Table of Contents

1. Executive Summary
2. Problem Statement
3. Motivation
4. Project Objectives
5. Target Users and Use Cases
6. Proposed Solution
7. System Architecture
8. Core Audit Pipeline
9. Dataset Validation Methodology
10. Health Score Methodology
11. Reporting System
12. Web Dashboard
13. CLI and Python API
14. Security and Privacy Model
15. Deployment and Distribution
16. Testing and Validation
17. Engineering Decisions and Design Principles
18. Limitations of v1.0.0
19. Future Development Directions
20. Expected Practical Impact
21. Conclusion

1. Executive Summary

ImageAudit is a local-first software platform for assessing the quality and structural integrity of image datasets used in computer-vision (CV) workflows. Version 1.0.0 is the first stable public release. It inspects datasets for integrity failures, image-quality issues, annotation defects (YOLO format), exact and perceptual duplicates, and cross-split leakage; emits structured findings; computes a transparent, weighted health score; and produces HTML, PDF, CSV, and JSON reports.

The system is designed for practitioners who need to diagnose dataset problems before training or evaluation, without uploading data to a remote service. Execution is local: a Python core engine powers a CLI, a Python API, a FastAPI service, and a Next.js dashboard. A Windows standalone executable packages the backend and a statically exported UI so end users need neither Python nor Node.

This proposal documents the problem context, solution design, architecture, validation methodology, security model, known limitations, and planned directions. Claims are limited to implemented capability and observed validation results for v1.0.0.

2. Problem Statement

Modern computer-vision systems depend heavily on the datasets used for training and evaluation. Defects in those datasets—corrupted files, inconsistent annotations, near-duplicate images across train and validation splits, class imbalance, and low-quality captures—are often discovered only after models underperform. At that point, debugging is expensive: training runs are long, failure modes are ambiguous, and root causes may lie in data rather than model architecture or hyperparameters.

Existing tooling tends to fragment across ad-hoc scripts, notebook checks, and framework-specific utilities. Many organizations lack a single, repeatable process that:

- systematically inspects dataset structure and file integrity;
- validates annotation formats against explicit rules;
- detects exact and perceptual duplicates, including cross-split leakage;
- produces machine-readable findings suitable for CI gates;
- and generates human-readable reports for review.

Additionally, datasets used in industrial, medical, or proprietary settings often cannot leave the local machine. Cloud-only quality platforms are therefore unsuitable for a significant class of users.

ImageAudit addresses this gap with a focused, local-first audit tool for YOLO-style detection datasets, emphasizing structured findings, transparent scoring, and practical distribution (CLI, API, web UI, and Windows executable).

3. Motivation

Dataset quality is a first-order determinant of CV system behavior. Integrity failures (truncated or unreadable images) break loaders and training loops. Annotation errors (invalid class indices, out-of-bounds boxes, empty or missing labels) inject noise into supervised learning. Duplicates within a split waste capacity and can bias metrics; duplicates across train and validation splits inflate reported performance and undermine claims of generalization.

Auditing datasets before training is therefore a form of risk reduction: it surfaces concrete, actionable defects early, when remediation is cheaper. A transparent health score—with documented components and weights—gives teams a shared language for dataset readiness without pretending that a single number captures every concern. Structured findings enable automation (for example, failing a continuous-integration job when critical defects are present).

Local-first execution matters for privacy, compliance, and air-gapped environments. Separating the audit engine from the web stack matters for maintainability and reuse: the same engine can drive CLI gates, notebooks, and the dashboard without duplicating analysis logic.

4. Project Objectives

The v1.0.0 objectives were scoped as follows:

ID	Objective	Status in v1.0.0
O1	Detect image integrity and quality issues on disk	Implemented
O2	Validate YOLO annotation structure and geometry	Implemented
O3	Detect exact and perceptual duplicates; flag cross-split leakage	Implemented
O4	Emit structured findings with severity and evidence	Implemented
O5	Compute a documented, configurable health score	Implemented
O6	Produce HTML, PDF, CSV, and JSON reports	Implemented
O7	Expose CLI, Python API, and HTTP API	Implemented
O8	Provide a local web dashboard with image explorer	Implemented
O9	Ship a Windows standalone executable with same-origin UI	Implemented
O10	Enforce path/archive security controls; no telemetry	Implemented
O11	Support additional dataset formats (e.g., COCO)	Not in v1.0.0 — future
O12	Semantic / embedding-based near-duplicate search	Not in v1.0.0 — future

5. Target Users and Use Cases

User group	Primary needs	Typical use of ImageAudit
CV engineers / ML engineers	Reliable training data; reproducible checks	CLI or API scan before training; CI validate gate
Dataset curators / label QA	Annotation correctness; visual review	Web dashboard, image explorer, HTML/PDF reports
MLOps / CI maintainers	Automated fail-on-severity policies	imageaudit validate in pipelines
Researchers (local data)	Privacy-preserving quality review	Local EXE or source install; offline operation
Teams without Python/Node	Simple desktop usage on Windows	ImageAudit.exe package

Representative use cases (supported by v1.0.0):

- Scan a YOLO dataset directory and obtain a health score plus findings before a training run.
- Fail a CI job when critical or high-severity findings exceed a configured threshold.
- Upload a zip archive of a dataset to the local API and review results in the browser.

- Export HTML/PDF reports for stakeholder review without exposing raw data off-machine.
- Delete audits and confirm cleanup of derived artifacts and extracted upload data.

6. Proposed Solution

ImageAudit implements a layered solution: a pure-Python audit engine at the center; adapters for dataset layouts; analyzers and detectors for specific checks; a scoring module; report generators; and multiple delivery surfaces (CLI, Python API, FastAPI, Next.js UI, Windows EXE).

The engine treats a dataset path as input, discovers samples and labels, runs integrity, quality, annotation, duplicate, and leakage analyses, aggregates statistics, produces findings, and computes the health score. Results can be persisted locally, queried via API, rendered in the UI, or written as reports. No component requires network access to external services.

The packaged Windows application embeds a static export of the frontend and serves it from the same FastAPI process, using same-origin `/api/...` requests so that development-time localhost URLs are not present in the production bundle.

7. System Architecture

The architecture separates concerns so that analysis logic does not depend on web frameworks, and the UI does not embed analysis code.

7.1 Logical layers

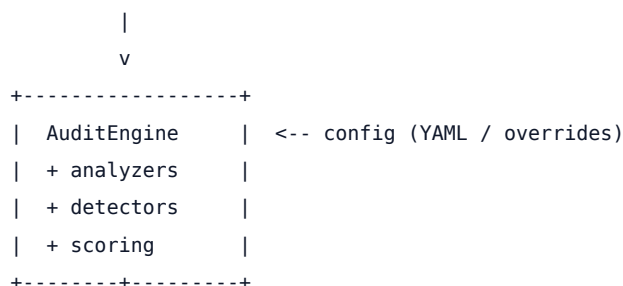
`backend/imageaudit/` contains the product core:

- core — engine orchestration, configuration, models, security helpers, paths, persistence store, measurement cache;
- datasets — layout discovery and YOLO adapter (extensible `DatasetAdapter` interface);
- analyzers — image integrity, quality metrics, annotation parsing, hashing;
- detectors — exact/perceptual duplicates and leakage classification;
- scoring — health score and finding construction;
- reports — JSON, CSV, HTML, PDF generators;
- api — service layer (unit-testable) and thin FastAPI route shell;
- cli — Click-based commands: scan, validate, report, serve, demo;
- desktop — packaged EXE entry point (loopback server + browser open).

The frontend (`frontend/`) is a Next.js App Router application. For packaging, it is built with static export (`NEXT_EXPORT=1`) into a directory bundled by PyInstaller and served by FastAPI's static-files mount.

7.2 Component interaction (textual overview)

[Dataset on disk / uploaded ZIP]



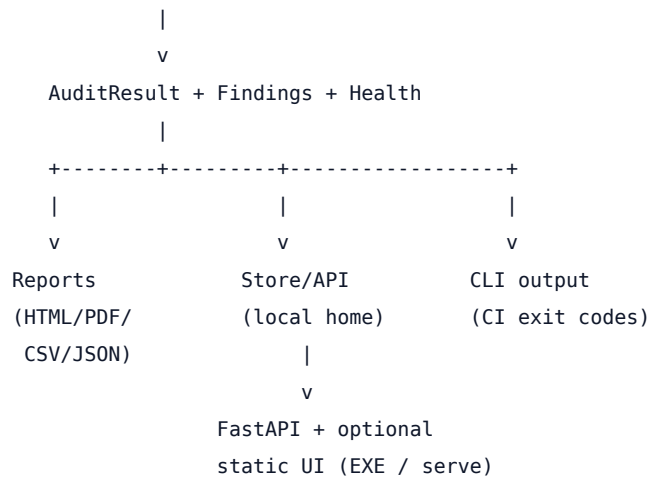


Figure 1. High-level data flow (logical).

7.3 Persistence and runtime layout

Writable state lives under a dedicated application home directory, not inside the dataset and not inside the PyInstaller extraction directory:

- Packaged Windows EXE: %LOCALAPPDATA%\ImageAudit (override with IMAGEAUDIT_HOME);
- Source / non-Windows: ~/.imageaudit or IMAGEAUDIT_HOME;
- Contents typically include audits, thumbnails, uploads, SQLite measurement cache, and logs.

Read-only packaged resources (static UI, report templates) are resolved from the frozen bundle root when running as an executable.

7.4 API boundary

HTTP routes in `api/app.py` translate requests into calls on `AuditService`. The service owns jobs, filtering, thumbnails, report generation, and deletion/cleanup. This boundary keeps route handlers thin and allows service-level unit tests without a full HTTP stack.

8. Core Audit Pipeline

A scan proceeds through staged work coordinated by `AuditEngine`:

Stage	Responsibility
Discover	Resolve dataset root; locate images and labels per supported YOLO layouts; parse optional <code>data.yaml</code>
Integrity	Readable decode, dimensions, format support, empty/truncated/corrupt detection
Quality	Blur (Laplacian variance), resolution, aspect ratio, brightness/contrast heuristics
Annotations	Line-level YOLO parse; class id range; box geometry and bounds; missing/empty/orphan labels
Duplicates	SHA-256 exact match; pHash/dHash perceptual match with configurable distances
Leakage	Classify duplicate groups that span train/val/test splits (exact vs perceptual)
Score & findings	Aggregate rates; compute component scores; emit severity-tagged findings
Persist / report	Optional store write; report generation on demand

Optional measurement caching reduces cost of repeated scans of unchanged files. Workers may be configured for parallel image analysis where applicable. Cancelled or failed jobs surface structured errors through the API and CLI.

9. Dataset Validation Methodology

9.1 Why audit before training

Training pipelines assume that files open, labels parse, and evaluation splits are independent. When those assumptions fail, symptoms appear as loader crashes, unstable loss, or suspiciously high validation metrics. Pre-training audits convert latent data risk into explicit findings that can be fixed or accepted with awareness. ImageAudit does not claim to improve model accuracy by itself; it aims to identify conditions that commonly undermine CV workflows.

9.2 Image integrity

Integrity checks verify that files are present, non-empty, and decodable as supported image types (JPEG, PNG, BMP, WebP, TIFF in v1.0.0). Truncated or corrupt payloads are flagged. Unsupported extensions are listed rather than silently ignored, so curators can see what the loader will skip. Dimension and channel metadata support downstream quality rules and reporting.

9.3 Image quality heuristics

Quality analysis applies deterministic heuristics: Laplacian variance for blur tendency, resolution thresholds, extreme aspect ratios, and brightness/contrast statistics. These are not perceptual quality models; they are practical flags for common acquisition failures. Thresholds are configurable because acceptable blur and resolution depend on the task.

9.4 YOLO annotation validation

YOLO detection labels are line-oriented. Malformed lines, non-numeric fields, class indices outside the declared set, coordinates outside [0, 1] (or otherwise invalid for the chosen convention), zero-area, tiny, huge, out-of-bounds, and duplicate boxes are reported with file and line context. Missing labels for images, empty label files, and orphan label files are first-class findings. Polygon (segmentation) lines are validated and reduced to bounding boxes for geometry checks; oriented boxes and keypoints are not interpreted in v1.0.0.

Annotation correctness matters because supervised detection loss trusts every box. Systematic label errors become systematic model errors, and they are cheaper to fix in the dataset than in post-hoc model debugging.

9.5 Duplicates and leakage

Exact duplicates use SHA-256 identity. Perceptual near-duplicates use pHash and dHash with configurable Hamming-distance thresholds and bucketing to avoid full pairwise cost on large sets. Groups are classified as within-split or cross-split. Cross-split groups are reported as leakage candidates (exact or perceptual).

Leakage matters because evaluation metrics assume held-out data. Near-identical images in train and validation can produce optimistic scores that will not hold on truly new data. ImageAudit's hash-based approach does not detect semantic similarity (same scene, different crop or adjacent video frames); that remains an explicit limitation and future direction.

9.6 Structured findings

Each finding carries severity, category, title, explanation, affected counts, examples, and recommended action. This structure supports human review and machine policy (for example, fail-on critical/high). Findings are preferable to unstructured log text for automation and triage.

10. Health Score Methodology

The health score is a weighted combination of component scores. Each component maps an observed defect rate (or imbalance measure) to a 0-100 score using a documented saturation function, so that zero defects yield a full component score and rates at or beyond a configured “zero_at” yield zero. Components that cannot be measured for a given dataset are excluded rather than treated as perfect.

Default components and relative emphasis (weights are configurable) include integrity, leakage, annotation validity, duplicates, annotation coverage, class balance, blur, and image statistics. Higher weights are assigned to integrity, leakage, and annotation validity because failures in those areas more directly break training or evaluation assumptions.

A transparent formula is preferred to an opaque single metric: teams can inspect which components drive a low score, adjust weights for domain needs, and avoid false confidence from a black-box “quality index.” The score is a hygiene indicator, not a predictor of model accuracy. Full documentation ships in the repository (docs/health-score.md).

11. Reporting System

Format	Role	Notes
JSON	Full machine-readable audit result	Suitable for archival and tooling
CSV	Tabular extracts (images, findings, issues, duplicates)	Spreadsheet-safe escaping for formula injection
HTML	Self-contained human report	Inline CSS/SVG; no scripts; no external network; escaped names
PDF	Printable stakeholder report	Generated via ReportLab

HTML reports are served with a restrictive Content-Security-Policy when delivered through the API. Report generation is available from the CLI, Python API, and dashboard. The design goal is reproducible artifacts that do not depend on a live server after generation.

12. Web Dashboard

The Next.js dashboard provides interactive access to the same audit data exposed by the API: overview and health breakdown, dataset summary, filterable image explorer with detail drawer and bounding-box overlay, duplicate groups, leakage views, findings tables, report export, scan progress, and settings. The UI does not embed demo data; numbers are fetched from the local API.

Because browsers cannot expose a dropped folder’s absolute path, folder scans are initiated by path on the server machine, while drag-and-drop accepts zip uploads to the local API only. In the packaged EXE, the UI is a static export served by FastAPI on loopback with same-origin API calls.

13. CLI and Python API

13.1 CLI

Primary commands include scan, validate (CI-oriented exit codes and thresholds), report, serve, and demo (synthetic sample generation). Validate supports policies such as fail-on severity and minimum health score, suitable for pipeline gates.

13.2 Python API

Library consumers can construct an AuditEngine with load_config, run a scan, inspect AuditResult, and write reports through the report generator. This path shares the engine with CLI and HTTP, preserving consistent behavior.

13.3 HTTP API

FastAPI exposes health, configuration defaults, scan submission (path or upload), job polling, audit listing and deletion, image pages, thumbnails, findings, duplicates, leakage, and report download endpoints. Contract documentation is maintained under docs/api.md.

14. Security and Privacy Model

ImageAudit is local-first. Dataset bytes are not uploaded to a vendor cloud as part of normal operation. There is no telemetry or analytics subsystem in v1.0.0.

14.1 Threat model (summary)

- Dataset paths and uploaded archives are untrusted input.
- The default network bind is loopback; there is no built-in authentication.
- The product is intended for local use; exposing the API beyond localhost requires additional hardening by the operator.

14.2 Controls implemented

- Path containment via resolve-and-parent checks; rejection of NUL bytes and empty paths; optional IMAGEAUDIT_ALLOWED_ROOTS allow-list.
- Zip extraction rejects absolute paths, traversal segments, drive markers, symlinks, and encrypted members; enforces limits on member count, member size, total uncompressed size, and compression ratio based on bytes written.
- Images are served by opaque audit/image identifiers, not client-supplied filesystem paths; reads re-check containment under the dataset root.
- Traversal-style audit identifiers are rejected; deletion removes audit records, thumbnails, and associated extracted upload trees without leaving orphaned audit files under normal operation.
- CORS is an explicit allow-list (development UI origins by default).
- HTML report responses include a restrictive CSP; template rendering uses autoescape.
- No shell=True / os.system usage in application paths reviewed for this release.

SECURITY.md in the repository documents supported versions, reporting process, and scope. Security claims are limited to these engineering controls; the project does not assert formal certification or exhaustive penetration-test coverage.

15. Deployment and Distribution

Mode	Audience	Requirements	Notes
Source (CLI/API/UI)	Developers	Python 3.11+, Node 18.18+ for UI	scripts/dev.ps1 or dev.sh
CLI-only	Automation	Python 3.11+	pip install backend package
Windows EXE	Desktop users	Windows x64	ImageAudit-v1.0.0-Windows-x64.zip
Docker Compose	Optional local stack	Docker	API + web; datasets mounted read-only

The Windows executable is built with PyInstaller from a static frontend export and the Python backend. The build pipeline can run packaging smoke tests against the produced binary. Runtime state uses %LOCALAPPDATA%\ImageAudit by default. Distribution of the EXE is via GitHub Releases (or equivalent), not as a Git-tracked binary.

16. Testing and Validation

Validation for v1.0.0 combines automated tests, static analysis, frontend checks, packaging, and an end-to-end Windows smoke test. Test counts describe executed checks; they are not a certificate of overall production readiness.

Layer	Result (v1.0.0 validation)
Backend unit/integration tests	173 passed, 2 skipped
Ruff (static analysis)	All checks passed
TypeScript typecheck	Passed
ESLint	Passed
Next.js production/static build	Passed
Windows PyInstaller one-file build	Succeeded (approx. 83.6 MB EXE in prior packaging notes)
Windows smoke test (packaged EXE)	Passed (UI, API, scan, reports, upload, persistence, deletion, restart, traversal rejection, cleanup)

Backend tests cover image validation, annotation parsing, duplicates/leakage, health math, reports, CLI behavior, path/archive security, persistence and deletion, and API service/routes. The packaging smoke test launches the real executable with an isolated data directory and exercises UI assets, same-origin API behavior, absence of development API URLs in bundled JS, and lifecycle operations listed above. A synthetic sample dataset with injected defects supports deterministic checks.

17. Engineering Decisions and Design Principles

Principle	Manifestation in ImageAudit
Separation of concerns	Engine independent of FastAPI/Next; service layer independent of routes
Local-first by default	Loopback bind; local store; no telemetry; optional root allow-list
Untrusted input handling	Path and archive guards; opaque IDs for image access

Principle	Manifestation in ImageAudit
Transparency	Documented health formula; structured findings; open configuration
Reproducibility	Deterministic sample dataset; config-driven thresholds; report artifacts
CI usefulness	validate command with severity/score policies and process exit codes
Packaging honesty	Static UI export; build fails if dev API URL leaks into bundle
Extensibility seam	DatasetAdapter interface for additional formats (not fully productized in v1.0.0)
Minimal dependency surface	Core analysis uses established scientific/imaging libraries; optional API extras

These choices prioritize maintainability and safe local operation over feature breadth. New dataset formats are expected to plug in at the adapter boundary without rewriting detectors or scoring, but shipping COCO or classification adapters is a future deliverable, not a v1.0.0 claim.

18. Limitations of v1.0.0

Area	Limitation
Dataset formats	YOLO detection first-class only; polygons reduced to boxes; no COCO/classification adapters yet
Annotation types	Oriented boxes and keypoints not interpreted
Duplicate detection	Hash-based only; no embedding/semantic similarity
Quality metrics	Heuristic thresholds; Laplacian blur also sensitive to dark/low-contrast images
Scale	JSON/on-disk audit store; very large datasets may need a different store/paging model
Platform packaging	One-file EXE is Windows-only; macOS/Linux users run from source or Docker
Authentication	None built-in; not intended for multi-user internet exposure
Model performance	Does not guarantee improved accuracy; diagnoses data hygiene issues only
UI polish	No formal accessibility audit claimed in this release

19. Future Development Directions

The following items are directions, not commitments or schedules. They extend the current architecture rather than replace it.

Direction	Rationale
COCO and classification adapters	Broaden format coverage via DatasetAdapter
Embedding-based near-duplicates	Catch semantic leakage beyond pHash/dHash
Stronger store and paging	Support substantially larger audits
Optional authentication modes	Safer shared-host deployments if required
Per-class quality metrics	Finer diagnostic detail for imbalanced problems
Dataset diff between audits	Track quality trends over curation iterations
Additional packaging targets	If demand justifies non-Windows desktop builds

20. Expected Practical Impact

Within its scope, ImageAudit is expected to help practitioners:

- detect concrete dataset defects earlier in the development cycle;
- standardize quality checks across CLI, CI, and interactive review;
- communicate dataset readiness using a shared, inspectable score and findings list;
- keep sensitive datasets on local infrastructure during quality review;
- distribute a simple Windows binary to users who cannot maintain a Python/Node toolchain.

Impact is operational and engineering-oriented. The project does not assert measured gains in published model benchmarks; those outcomes depend on how teams act on findings.

21. Conclusion

ImageAudit v1.0.0 delivers a coherent, local-first toolchain for auditing YOLO-oriented computer-vision datasets. Its primary engineering contribution is the combination of a reusable audit engine, structured findings, a transparent health score, multi-format reporting, and multiple delivery surfaces — including a security-conscious Windows package with same-origin UI and API — without requiring cloud upload.

The release is validated by automated backend tests, frontend build checks, and a full Windows executable smoke test covering scan, reporting, upload, persistence, and cleanup paths. Limitations are explicit: format coverage, hash-only near-duplicates, local-trust assumptions, and Windows-only one-file packaging among them.

As a v1.0.0 engineering release, ImageAudit is intended to be useful immediately for local dataset hygiene and CI gating, while providing a clear architecture for format extensions and deeper similarity analysis in future work.

Repository: <https://github.com/AmirSam-Eftekhari/ImageAudit>

License: MIT · Version: 1.0.0